Lingua: English **Italiano** Українська

Guida h4nb

N-body diretto su GPU NVIDIA: Hermite del 4° ordine, passi temporali a blocchi, FP64

> ✅ = funziona ora su `main`. 🚧 = progetto concordato, non ancora implementato (numero del task tra parentesi).

Indice

1. Il problema
2. L'approccio
3. Architettura
4. Avvio rapido
5. Configurazione
6. Promemoria
7. Problemi comuni
8. Hardware
9. Documenti

1. Il problema

Un ricercatore studia un disco o un ammasso attorno a un corpo massiccio: nubi di gas attorno a un nucleo galattico attivo (AGN), oppure stelle attorno a una massa centrale. Le particelle possono avere dal 10 % al 100 % della massa centrale, quindi il corpo centrale si muove.

- **Forze esatte:** ogni particella attrae tutte le altre. Un calcolo completo delle forze con N = 1 024 000 sono circa 10^{12} coppie.
- **Precisione controllata:** l'errore relativo massimo dell'energia su un periodo orbitale medio è al massimo 1e-8. Lo deve dimostrare un'esecuzione misurata.
- **Velocità:** le simulazioni grandi terminano in ore su GPU da data center.
- **Riproducibilità:** stesso input, configurazione, dispositivi e build danno lo stesso output, bit per bit.

2. L'approccio

Tema	Decisione	Motivo
Integratore	Hermite del 4° ordine: predizione, calcolo di a e del jerk, correzione	4° ordine con un solo calcolo delle forze per passo
Passi temporali	Criterio di Aarseth (η), passi a blocchi in potenze di due, tick interi	Solo le particelle attive ricevono nuove forze; tempi dei blocchi esatti
Corpo centrale	Particella 0, partecipante a pieno titolo	Conserva la quantità di moto
Precisione	Solo FP64, niente fast-math, niente atomici float, ordine di somma fisso	Precisione ed esecuzioni identiche bit per bit

Tema	Decisione	Motivo
Softening	Plummer ε opzionale; preset AGN 0.01, stellare 0.0001	Cambia il modello; non nasconde gli errori
Più GPU	Sorgenti suddivise (j-parallel); il proprietario somma i parziali in ordine	Poche particelle attive per passo; stesso lavoro per GPU
Più nodi ✅ (compilato; non provato su GPU)	Un rank MPI per GPU, MPI CUDA-aware, somma del proprietario in ordine di rank (T029–T036)	Obiettivo: 4–8 GH200
η	Misurarlo per ogni sistema; circa 5e-4 per 1e-8	L'intervallo 0.001–0.02 del TZ non raggiunge 1e-8
Ottimizzazione	Ogni ottimizzazione richiede una misura che ne mostri il bisogno	Le stime di velocità su GPU sono spesso sbagliate

3. Architettura

Un solo header pubblico (`include/h4nb/h4nb.hpp`) e una sola classe, `Simulation` . Nessun tipo CUDA nell'API. I kernel GPU e l'oracolo CPU dei test condividono lo stesso header matematico (`src/hermite.cuh`).

Un passo a blocchi:

1. Trovare il prossimo tempo di blocco Tn = min(next).
2. Predire tutte le particelle a Tn.
3. Selezionare le particelle attive (next == Tn).
4. Inviare i target attivi a ogni GPU.
5. Ogni GPU calcola a e jerk parziali delle proprie sorgenti su tutti i target (tile in shared memory, un thread per target).
6. Inviare ogni porzione alla GPU proprietaria.
7. Il proprietario somma i parziali nell'ordine dei dispositivi, corregge e sceglie il nuovo passo.

Memoria: circa 0,4 GB per GPU con 1M particelle. Il limite è il calcolo.

4. Avvio rapido

Build (Linux x86_64 o aarch64, CUDA ≥ 12.0, CMake ≥ 3.24, GPU con compute capability ≥ 7.0):

```
make build                                  # Release, CMAKE_CUDA_ARCHITECTURES=native
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=89      # RTX 40
70 Ti
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=90      # GH200
/ H100
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES="89;90" # both
cmake --build build -j
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=90 -DH4NB_MPI=ON
# MPI (several nodes)
```

Esecuzione:

```
build/h4nb_mkdisk n=127999 frac=0.1 out=disk128k.snap     # prints mean / median / mass-we
ighted period
build/h4nb_run ic=disk128k.snap out=run128k t_end=6.3125 diag=0.0625 eta=0.0005 devices=0,
1
build/h4nb_order e=0.5                                      # Kepler orbit + 4th-order chec
k
build/h4nb_bench sizes=16384,131072,1024000 tiles=64,128,256 bpsm=2,4,8 devices=0,1   # be
nch/bench.dat
build/h4nb_study ic=disk128k.snap out=study diag=0.0625 etas=0.001,0.0005,0.00025 devices=
0,1   # eta study
tools/h4nb_pack.sh run128k study                             # pack logs + results -> h4nb_
logs_<host>_<time>.tar.gz
```

`t_end`, `diag` e `snap` devono essere multipli di `dt_max` (predefinito 1/64), e `diag` deve dividere `t_end`.

5. Configurazione

✅ Chiavi di `h4nb_run` (file, ambiente o riga di comando, `key=value`):

Chiave	Predefinito	Significato
`ic`	—	condizioni iniziali (testo o .snap)
`out`	—	cartella di output
`t_end`	—	tempo finale
`eta`	`0.01`	precisione Aarseth (≈0.0005 per 1e-8)
`eta_init`	`0.01`	fattore del primo passo
`dt_max`	`1/64`	passo massimo (livello 0)
`max_level`	`40`	dt_min = dt_max / 2^max_level
`eps`	`0`	lunghezza di softening ε
`mode`	`block`	block o fixed
`fixed_level`	`0`	livello in modalità fixed
`fixed_center`	`0`	1: il centro non si muove (test)
`devices`	`0`	elenco GPU, es. 0,1
`diag`	`dt_max`	intervallo dei record di energia
`snap`	`0`	intervallo degli snapshot (multiplo di diag)
`r_close`	`-1`	raggio coppie vicine; 0 = spento
`n`	`0`	numero atteso di particelle (0 = nessun controllo)
`central_id`	`0`	ID del record 0 (corpo centrale)
`period`	`0`	periodo di riferimento (0 = stima, richiede G = 1)

Chiave	Predefinito	Significato
`period_kind`	`mean`	mean, median o mass_weighted
`preset`	`none`	none, agn (ε 0.01), stellar (ε 0.0001)
`G`	`1`	costante gravitazionale
`dissipation`	`off`	off o linear_drag: a_diss = −(v − v_bg)/τ, non sulla particella 0; il verdetto usa E − E_0 − W_diss
`drag_tau`	`0`	tempo di attrito τ, > 0 con linear_drag
`drag_vbg`	`0,0,0`	velocità di fondo v_bg (3 valori)
`tile`	`128`	target per blocco: 64, 128 o 256
`blocks_per_sm`	`4`	da 1 a 16
`snap_times`	—	tempi espliciti degli snapshot
`energy_tol`	`1e-8`	soglia del verdetto sull'energia
`perf`	`1`	0: niente performance.dat
`device_offset`	`0`	solo MPI: GPU = (rank locale + device_offset) % numero di GPU
`mpi_host_staging`	`0`	solo MPI: 1 = buffer MPI copiati tramite memoria host pinned (MPI non CUDA-aware)

✅ Cinque sorgenti. Vince la sorgente successiva:

1. valore predefinito
2. `preset=none|agn|stellar`
3. `config=<file>` (righe key=value)
4. variabili d'ambiente `H4NB_<KEY>`
5. riga di comando

Una chiave sconosciuta o un valore errato è un errore (codice di uscita 2). Ogni esecuzione scrive `config.used`. `help=1` stampa tutte le chiavi.

Come scegliere i valori principali

`eta`	Misurarlo con lo studio di η. Partire da circa 0.0005. Il costo cresce come 1/√η.
`dt_max`	Da 1/64 a 1/16 del periodo più breve di interesse.
`max_level`	Aumentarlo se compare l'avviso dt < dt_min.
`eps`	0 per masse puntiformi; preset=agn o preset=stellar per i due tipi di modello.
`diag`	Un controllo dell'energia costa $N^2/2$ coppie. Con 1M usare T/16 o più.
`tile, blocks_per_sm`	Misurarli con h4nb_bench; salvare i valori migliori nella config della macchina.
`GPU / nodi`	Li imposta il launcher (devices=, mpirun, srun). La dimensione del cluster non è fissata nel codice.

6. Promemoria

File di configurazione ✅

```
# configs/agn.cfg
preset=agn                    # eps = 0.01
eta=0.0005                    # measured value for 1e-8
dt_max=0.015625
diag=0.0625
period=6.283185307179586      # 2*pi for G = 1, M = 1, a = 1
```

Esecuzione con file di configurazione ✅

```
build/h4nb_run config=configs/agn.cfg ic=disk128k.snap out=run1 t_end=6.3125 devices=0,1
H4NB_ETA=0.00025 build/h4nb_run config=configs/agn.cfg ic=disk128k.snap out=run2 t_end=6.3
125
build/h4nb_run config=run1/config.used out=run1_repeat     # repeat a run exactly
```

Più nodi (MPI) ✅ compilato, prova su GPU da fare

```
# build with -DH4NB_MPI=ON; one rank per GPU; only rank 0 writes files
srun --ntasks-per-node=4 --gpus-per-task=1 --gpu-bind=single:1 \
  build/h4nb_run config=configs/gh200.cfg ic=disk1m.snap out=run1m t_end=6.3125
mpirun -np 8 --map-by ppr:4:node --bind-to core \
  build/h4nb_run config=configs/gh200.cfg ic=disk1m.snap out=run1m t_end=6.3125
mpirun -np 2 build/h4nb_run ic=disk.snap out=r2 t_end=6.3125 mpi_host_staging=1   # MPI no
t CUDA-aware
mpirun -np 4 build/h4nb_run ic=disk.snap out=r4 t_end=6.3125 device_offset=4      # GPUs 4
-7 of a node
# exit codes on all ranks: 1 runtime, 2 settings/input, 3 NaN; CUDA/MPI error: MPI_Abort
```

C++ ✅

```
#include <cmath>
#include <h4nb/h4nb.hpp>

h4nb::Particles p = h4nb::read_snapshot("disk128k.snap");   // or h4nb::load_text("ic.tx
t")
h4nb::Config c;
c.eta = 5e-4;  c.dt_max = 1.0 / 64;  c.eps2 = 0.01 * 0.01;  c.devices = {0, 1};
h4nb::Simulation sim(p, c);
const auto e0 = sim.diagnostics();
sim.advance_to(6.3125);                                      // a multiple of dt_max
const auto e1 = sim.diagnostics();
double rel = std::abs((e1.E - e0.E) / e0.E);                 // the 1e-8 goal
if (sim.status() & h4nb::kNaN) { /* stop */ }
```

File di output ✅

`energy.dat`	`time K U E rel_err max_rel_err mean_rel_err W_diss`
`energy_periods.dat`	`period t_start t_end max_rel_err mean_rel_err complete`
`study/study.dat`	`eta dt_max t_end max_rel_err ... pair_evals wall_s pass`
`momentum.dat`	`time Px Py Pz Lx Ly Lz dP_rel dL_rel`
`timesteps.dat`	`time dt_min dt_max dt_mean mean_active r_min`
`performance.dat`	`time wall_s block_steps active pairs_per_s`
`bench/bench.dat`	`N devices tile blocks_per_sm pairs_per_s gflops wall_s [pct_peak]`
`snap_NNNNNN.snap`	`binary: header + id m x y z vx vy vz`
`config.used`, `run_info.txt`	`configuration; commit, CUDA, GPUs, host, command`

7. Problemi comuni

Sintomo	Causa	Azione
max |dE/E| > 1e-8	η troppo grande	Ridurre eta (provare 0.0005, poi 0.00025). Eseguire lo studio di η.
warning: dt < dt_min	Una particella richiede un passo più fine	Aumentare max_level. Controllare le coppie vicine.
warning: pair closer than r_close	Incontro ravvicinato	Controllare r_min in timesteps.dat. Valutare il softening se il modello lo consente.

Sintomo	Causa	Azione
non-finite value (NaN or Inf)	Input errato o collisione	Controllare l'input. L'esecuzione si ferma per progetto.
advance_to: t must be a multiple of dt_max	t_end, diag o snap fuori griglia	Usare multipli di dt_max (1/64 = 0.015625).
La build fallisce con arch native	Nessuna GPU sulla macchina di build	Impostare CMAKE_CUDA_ARCHITECTURES=89 o 90.
Codice di uscita 2 e un messaggio key=value	Chiave sconosciuta, valore errato o fuori intervallo	Leggere il messaggio: indica chiave, valore e sorgente. Eseguire help=1.
Esecuzione 1M molto lenta su RTX	FP64 lento (1/64)	Usare RTX per N ≤ 131 072; GH200 per 1M.
Risultati diversi con numero di GPU diverso	Ordine di somma diverso	Previsto. Confrontare con la tolleranza scritta, non bit per bit.

8. Hardware

GPU	Uso	Nota
RTX 4070 Ti (8.9)	Sviluppo, N ≤ 131 072	≈17 h per un periodo con 1M (stima)
GH200 (9.0)	Esecuzioni 1M e di produzione	≈18 min per un periodo con 1M su una GPU (stima)
Sotto 7.0 (GTX 1060)	Non supportata	Costituzione 2.0.0

9. Documenti

- README — `README.md`
- Constitution — `.specify/memory/constitution.md`
- Spec — `specs/001-hermite-nbody-lib/spec.md`
- Plan — `specs/001-hermite-nbody-lib/plan.md`
- Research — `specs/001-hermite-nbody-lib/research.md`
- Run configuration contract — `specs/001-hermite-nbody-lib/contracts/run-config.md`
- Tasks — `specs/001-hermite-nbody-lib/tasks.md`
- Quickstart — `specs/001-hermite-nbody-lib/quickstart.md`
- Multi-node design — `docs/multi-node-design.md`
- CUDA best practices — `docs/cuda-best-practices.md`

Generato dai documenti del repository con docs/guide/build_guide.py. build_guide.py